grep, sed, awk

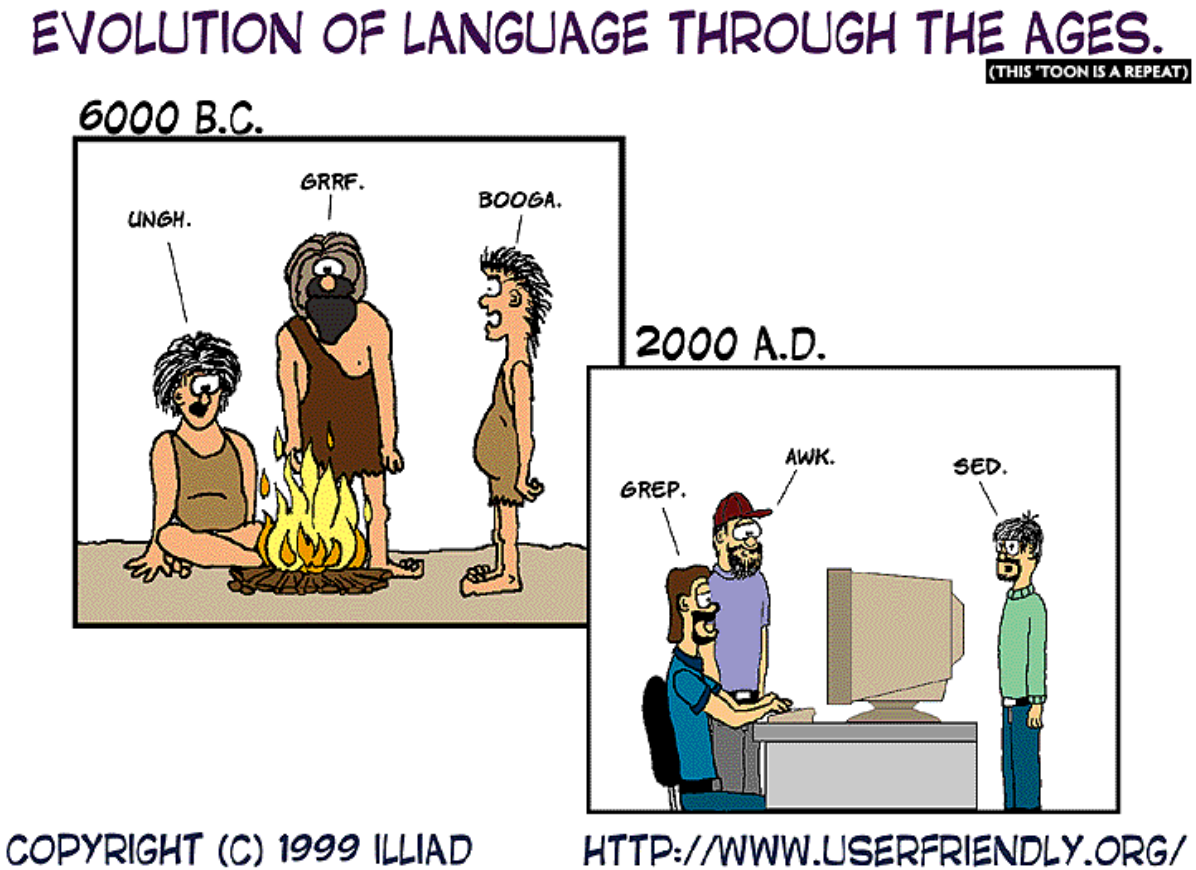
Benedict Reuschling

December 1, 2025



Overview

In this part of the lecture we will look at `grep`, `sed`, and `awk` as tools for processing and analyzing of data.



Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

The grep Program

The grep command (abbrev. for **g**lobal/**r**egular **e**xpression/**p**rint) searches for lines in files or standard input that match a specific pattern and outputs it by default. The syntax of grep is:

```
grep Pattern File
```

Use a single character, a string, or a regular expression (i.e. *, ?) for Pattern.

Example: Search for the swap file in /etc/fstab:

```
$ grep swap /etc/fstab
/dev/da0s1b  none  swap sw  0 0
```

grep is often used to search log files for specific entries. grep(1) includes helpful options.

Variants of grep with Examples

There are other versions grep in existence, some shown here:

fgrep fast grep. The pattern to search for are constant strings without regular expressions, which improves search time somewhat.

egrep extended grep. Better algorithm for finding patterns and allows extended regular expressions.

Usage

Search all files in the current directory that contain the main(string:

```
$ grep -l "main(" *.c
prog.1.1.c
prog.1.c
```

Example Usage of grep 1/3

Search all lines that end in `n` or `N` (`-i`: case insensitive):

```
$ cat file1
This is a line with n
This is a line with n.
This is a line with N
This is a line with N$
$
$ grep -i "n$" file1
This is a line with n
This is a line with N
$
```

Search all lines that begin with `T` or `t`:

```
$ cat file1
This is a line with n
this is a line with n.
A line with N
Another line
$
$ grep -i "^T" file1
This is a line with n
this is a line with n.
```

Example Usage of grep 2/3

All lines that contain a non-empty sequence of . (dots). Compare egrep and grep output:

```
$ cat file2
Line without a dot
Line with a dot.
Line with sequence of dots ......
$
$ egrep "(\.)+" file2
Line with a dot.
Line with sequence of dots ......
$
$ grep "(\.)+" file2
$
```

All lines that begin with no more than two numbers:

```
$ cat file3
Line1
2. line
12 last line
$
$ egrep '^[0-9][0-9]' file3
12 last line
$
```

Example Usage of grep 3/3

Edit files with `vi` that contain `printf`:

```
$ vi $(grep -l printf *.c)
```

This example generates a list of files, not their content. Upon finding the string the search continues in the next file.

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

The awk Program

Search files line by line with grep. What to do when searching columns and echo them to the screen? For that, use awk.
awk is a filter for manipulating text files written by

- Alfred V. **A**ho
- Peter J. **W**einberger
- Brian B. **K**ernighan

as a programming language for processing and analyzing data.
The following **features** are characteristic for awk:

- Simple, yet **powerful programming tool**,
- specializing in **ASCII data**,
- **short** awk-**programs** and
- written in (**rapid prototyping**) (quick) style with a
- **C**-like **syntax**.

The awk Program

Specific separators allows awk to separate columns (default: space). `$i` represents column i, $0 all columns.
Example: Search for the swap file in `/etc/fstab` and extract the device (1. column):

```
$ grep swap /etc/fstab | awk '{ print $1 }'
/dev/da0s1b
```

awk Example Table

Use the following table throughout the next slides to illustrate the functionality of awk:

```
$ cat soccerleague
Munich - Rome                   3 : 2 34000 Visitors
Milan - Madrid                  2 : 1 28260 Visitors
Liverpool - Juventus            3 : 0 10000 Visitors
Barcelona - Real                0 : 0 20600 Visitors
Porto - Manchester              1 : 0 22000 Visitors
Marseille - Celtic              2 : 0 34000 Visitors
Chelsea - Valencia              0 : 1 21000 Visitors
$
```

awk Examples for Comparing Columns

Who won as home team?

```
$ awk '$4 > $6 { print $1 }' soccerleague
Munich
Milan
Liverpool
Porto
Marseille
$
```

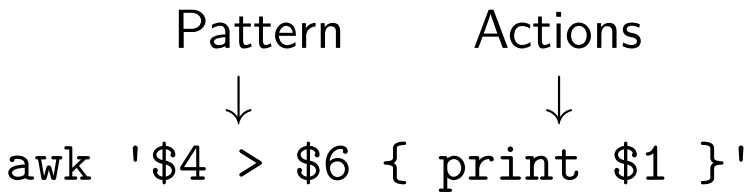
Which matches ended in a draw?

```
$ awk '$4 == $6 { print $1, "-", $3, $4 ":" $6 }' soccerleague
Barcelona - Real 0:0
$
```

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Program Structure and Call Syntax



Pattern and action are both optional.
pattern without action (whole line returned):

```
$ awk '$4 > 2' soccerleague
Munich - Rome 3 : 2 34000 Visitors
Liverpool - Juventus 3 : 0 10000 Visitors
$
```

Action without pattern (all lines processed):

```
$ awk '{ print $1 " shot " $4 " goals at home" }' soccerleague
Munich shot 3 goals at home
Milan shot 2 goals at home
Liverpool shot 3 goals at home
Barcelona shot 0 goals at home
Porto shot 1 goals at home
Marseille shot 2 goals at home
Chelsea shot 0 goals at home
```

Program Structure and Call Syntax

`awk` called as follows:

```
awk 'awk-program' file(s)
```

```
$ awk '$4 > $6 { print $1 }' soccerleague1 soccerleague2
...
```

Without any input files specified awk uses **standard input** as a file:

```
$ awk '$1 > 0 {print $1, " with tax: ", $1*1.19}' 100
100 with tax: 119
$
```

Program Structure and Call Syntax

An awk program stored in a **file**:

```
$ cat valueaddedtax.awk
$1 > 0 {print $1, " with value-added tax: ", $1*1.19} $
$ awk -f valueaddedtax.awk
100
100 with value-added tax: 119
```

awk call as a filter:

```
$ echo 100 | awk -f valueaddedtax.awk
100 with tax: 119
```

```
$ echo 100 | awk '$1 > 0 {print $1, " with tax: ", $1*1.19}'
100 with tax: 119
```

awk call with **program** and **data** files:

```
$ awk -f valueaddedtax.awk sums
100 with tax: 116
200 with tax: 232
300 with tax: 348
```

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Simple and Formatted Output

Display **every line**:

```
$ cat soccerleague
Munich - Rome                    3 : 2 34000 Visitors
Milan - Madrid                   2 : 1 28260 Visitors
Liverpool - Juventus             3 : 0 10000 Visitors
Barcelona - Real                 0 : 0 20600 Visitors
Porto - Manchester               1 : 0 22000 Visitors
Marseille - Celtic               2 : 0 34000 Visitors
Chelsea - Valencia               0 : 1 21000 Visitors
$
```

Return **certain fields with separators**:

```
$ awk '{print $1 $3}' soccerleague
Munich Rome
Milan Madrid
Liverpool Juventus
Barcelona Real
Porto Manchester
Marseille Celtic
Chelsea Valencia
$
```

Simple and Formatted Output

Return **certain fields without separators**:

```
$ awk '{print $1$3}' soccerleague
MunichRome
MilanMadrid
LiverpoolJuventus
BarcelonaReal
PortoManchester
MarseilleCeltic
ChelseaValencia
$
```

Text Output:

```
$ awk '$6 > $4 { print $3 " wins against " $1 }' soccerleague
Valencia wins against Chelsea
$
```

Simple and Formatted Output

Using awk, **printf**-like **formatted output** is possible:

```
$ awk '{printf("%-10s: %6d Visitors; %3d goals\n",
 $1, $7, $4+$6)}' soccerleague
Munich     :   34000 Visitors;     5 goals
Milan      :   28260 Visitors;     3 goals
Liverpool  :   10000 Visitors;     3 goals
Barcelona  :   20600 Visitors;     0 goals
Porto      :   22000 Visitors;     1 goals
Marseille  :   34000 Visitors;     2 goals
Chelsea    :   21000 Visitors;     1 goals
$
```

The `printf` characters are the same as in the C function `printf`.

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Comparisons

Comparisons **in a pattern**:

```
$ awk '$4 > $6 { print $1 }' soccerleague
Munich
Milan
Liverpool
Porto
Marseille
$
```

Comparison with calculation:

```
awk '$4+$6 > 3 { printf("More than 3 goals shot in %s\n", $1)}'
 soccerleague
More than 3 goals shot in Munich
$
```

Text comparisons:

```
awk '$1=="Munich" {printf("%d goals in Munich\n", $4+$6)}'
 soccerleague
5 goals in Munich
$
```

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Begin and End Actions

Specify actions executed once at the beginning or end as follows:

```
BEGIN { Actions }
      {...}
END { Actions }
```

```
$ cat table.awk
BEGIN { printf("%10s - %-10s | %s\n","Home", "Guest", "Result") }
{ printf("%10s - %-10s | %2d : %2d\n", $1, $3, $4, $6); }
END { printf("-----------------------------------------\n") }
$
```

```
$ awk -f table.awk soccerleague
      Home - Guest           | Result
    Munich - Rome            |   3:2
     Milan - Madrid          |   2:1
 Liverpool - Juventus        |   3:0
 Barcelona - Real            |   0:0
     Porto - Manchester      |   1:0
 Marseille - Celtic          |   2:0
   Chelsea - Valencia        |   0:1
-----------------------------------
```

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Variables, Types, and Expressions

Predefined Variables

NF - Number of fields in the current input line

```
$ awk '{ print NF, $1, $(NF-1), $NF }' soccerleague
8 Munich 34000 Visitors
8 Milan 28260 Visitors
8 Liverpool 10000 Visitors
8 Barcelona 20600 Visitors
8 Porto 22000 Visitors
8 Marseille 34000 Visitors
8 Chelsea 21000 Visitors
$
```

NR - The current line number

```
$ awk '{ print "Match", NR, $1 $2 $3 }' soccerleague
Match 1 Munich-Rome
Match 2 Milan-Madrid
Match 3 Liverpool-Juventus
Match 4 Barcelona-Real
Match 5 Porto-Manchester
Match 6 Marseille-Celtic
Match 7 Chelsea-Valencia
```

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Variables, Types and Expressions

User Defined Variables

awk creates and initializes variables automatically upon first use.

```
$ cat guestwinners.awk
$6 > $4 { guestwin = guestwin + 1 }                    ← Variable definition
END { print guestwin," guest(s) won" }
$
$ awk -f guestwinners.awk soccerleague
1 guest(s) won
$
```

```
$ cat matchanalysis.awk
{ sum = sum + $7 }
END {
    print NR, "matches"
    print "Total visitors:", sum
    print "Average per match:", sum/NR
}
$
$ awk -f matchanalysis.awk soccerleague
7 matches
Total visitors: 169860
Average per match: 24265.7
```

Variables, Types and Expressions

User Defined Variables

A calculation of biggest values is easy with **sub-actions**:

```
$ cat mostVisitors.awk
$7 > max { max =$7; city =$1 }
END { print "Most visitors were in ", city, "(" max ")" }
$
$ awk -f mostVisitors.awk seccerleague
Most visitors were in Munich (34000)
$
```

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Passing Variables from the Shell to awk

Pass variables from the shell or a shell script to awk for further processing using the -v parameter:

```
$ cat singleclub.sh
#!/bin/sh
echo "What is your favorite club?"
read club
awk -v c=$club '{ if ($1 == c || $3 == c) print $0; }' soccerleague.txt
$ ./singleclub.sh
What is your favorite club?
Munich
Munich - Rome                    3 : 2 34000 Visitors
```

In this example, we prompt the user for a specific club on the shell and then pass that variable to awk using -v (variable, not verbose) to filter out that specific club. Any number of variables passed this way by providing the -v parameter each time.

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Variables, Types and Expressions

User Defined Variables

Create a string variable clubs step by step.

```
$ cat lossesasguests.awk
BEGIN { print "The following teams lost as guests:" }
$4 > $6 { clubs = clubs $3 " " }
END { print clubs }
$
$ awk -f lossesasguests.awk soccerleague
The following teams lost as guests:
Rome Madrid Juventus Manchester Celtic
$
```

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Types and Expressions

In awk the types *number* (always float) and *string* are possible. The usage context of a variable or the field type in the file determines the variable type.
When two variables are numerical then comparisons will also be numerical, otherwise lexical.

`NF > 5` is true, when there are more than 5 fields present.

`$1 > "s"` is true, when the first field comes after s lexicographically (i.e. "susan").

Using a variable as a **number**, **add 0**; when using a variable as a string **append the null string**.

```
a = 12          ← a has the value 12.0
b = a " "       ← b is now 12 not "12.0"
```

To form expressions, use most of the operators known from C (from highest to lowest priority).

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Control Structures - if

The following control structures are available:

```
if ( expression ) statement else statement
```

```
$ cat toto.awk
BEGIN { print "The toto numbers are:" }
    { if ($4 > $6)
        printf("1 ")
    else if ($4 == $6)
        printf("0 ")
    else
        printf("2 ")
    }
END { printf("\n") }
$
$ awk -f toto.awk soccerleague
The toto numbers are:
1 1 1 0 1 1 2
$
```

Control Structures - for

```
for ( expression; expression; expression ) statement
```

```
$ cat grades.awk
# Grade calculation
# Input: Name grade1 grade2 grade3 ....
# Output: Student: Final grade
{ sum = 0
for (i=2 ; i<=NF ; i=i+1)
    sum = sum + $i
    printf("%20s: %2.2f\n", $1, sum/(NF-1))
}
$
$ cat examresult
Eve  1.1 1.2 3.3
Adam 3.0 2.7 5.0
$
$ awk -f grades.awk examresult
Eve: 1.87
Adam: 3.57
$
```

Control Structures - while

```
while ( expression ) statement
do statement while ( expression )
```

```
$ cat investments
BankA 2010 100000 4 2012
BankB 2009 100000 6 2012
$
$ cat interest.awk
# calculate future interest of an investment
# Input: Name InvestmentYear Amount Interest(%) EndYear
# Output: Amount of money at the end of the last year
{
    printf("\n%s:\n", $1);
    i = $2
    amount = $3
    interest = $4/100
    while (i<=$5) {
        printf("\t%4d : %10.2f EUR\n", i, amount);
        amount = amount * (1 + interest)
        i++
    }
}
$
```

Control Structures - while

```
while ( expression ) statement
do statement while ( expression )
```

```
$ awk -f interest.awk investments
BankA:
2010 :   100000.00 EUR
2011 :   104000.00 EUR
2012 :   108160.00 EUR
BankB:
2009 :   100000.00 EUR
2010 :   106000.00 EUR
2011 :   112360.00 EUR
2012 :   119101.60 EUR
```

Besides, `break` and `continue` known from C are also available.

Functions

Functions are available in `awk` like

- numerical calculations (`sin`, `cos`, ...)
- string functions (`index`, `split`, ...)
- time and date (`strftime`, ...)
- bit manipulations (`shift`, ...)

User defined functions are also possible:

```
function name(parameter list) { statements }
```

Functions

Example (Factorial of a number):

```
$ cat factorial.awk
# factorial
function factorial( n ) {
    if (n <= 1) return 1
    else return n * factorial(n - 1)
}
$1 > 0 { print "the factorial of ", $1, "is ", factorial($1) }
$
```

```
$ awk -f factorial.awk
3
the factorial of 3 is 6
10
the factorial of 10 is 3628800
100
the factorial of 100 is 9.33262e+157
$
```

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Regular Expressions

Instead of a pattern specify a regular expression. With that, an `awk` program has the following form:

```
Pattern { Actions }
```

or

```
/ regular expression / { action }
```

Display lines that match a specific pattern:

```
$ awk '/^#include/{print}' stdio.h
```

Regular Expressions

Print **users in a Unix system**, that use bash as their default shell:

```
$ cat bashUser.awk
BEGIN {
    FS=":";
    print "Bash users:"
    print "-----------------------------------------";
    printf "%-15s %-30s %s\n", "user-id", "Name", "Home";
    print "-----------------------------------------";
}
/\/bin\/bash/ {printf "%-15s %-30s %s\n", $1, $5, $6}
$
$ awk -f bashUser.awk /etc/passwd | more
Bash users:
-----------------------------------------
user-id         Name            Home
-----------------------------------------
sue             Sue Sunshine    /home/sue
bob             Bob Hopefully   /home/bob
...
$
```

awk instead of grep

With the knowledge about this regular expression syntax, replace common pipes from grep to awk. Consider a common pipe construct like this:

```
$ cal | grep December | awk '{ print $2 }'
2017
```

Shortened to one pipe by omitting grep and using the awk regex syntax:

```
$ cal | awk '/December/ { print $2 }'
2017
```

The inverted grep (grep -v) to print out lines that do *not* match, works like this:

```
$ cal | awk '! /December/ { print $2 }'
...
```

This construct is shorter to type and spawns one less process. To process the input further awk offers even more options, so we gain greater flexibility.
Source: http://blog.jpalardy.com/posts/skip-grep-use-awk/

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

The sed Program

Use the sed (**s**tream **ed**itor) program to manipulate text streams. It can make changes to text files via the command line in an automatic fashion. To do that, sed reads a file or a text stream line by line as input and applies user-defined rules to it. The changed stream is then output and redirected into another file.
In the chapter about vi, we already used sed rules (without knowing it) when we were using search and replace. The simplest form of sed is:

```
s/searchstring/replacementstring/
```

In this case, look for searchstring and replace it with replacementstring when found.

Manipulating Textfiles with sed

When using sed to change the contents of a file, *do not* specify the input file as output file. Otherwise, the inputfile will be empty when sed finished processing. Instead, redirect the output into a new file.

Example:

```
$ cat firstnames.txt
susi
Hans
Peter
Anna
$
$ sed 's/susi/Susi/g' firstnames.txt > firstnamesCaps.txt
$ cat firstnamesCaps.txt
Susi
Hans
Peter
Anna
```

The file `firstnames.txt` remains unchanged. The option `s` at the beginning stands for **s**ubstitute and the `g` at the end of the statement for **g**lobal, to match all occurrences within the file. Without it, it replaces the first match.

Usage of sed

Pipe strings to sed directly:

```
$ echo "We're open today" | sed s/open/closed/
We're closed today
```

When searching and replacing the / (slash) itself, quoting is one option:

```
$ echo "C:/TEMP" | sed 's/\//\\/'
C:\TEMP
```

or use a different separator by directly passing it after the s:

```
$ echo "C:/TEMP" | sed 's:/:\\:'
C:\TEMP
```

Note: there must always be 3 separators present. Otherwise, sed will return the following error:

```
sed: 1: "s:/:\\": unterminated substitute in regular expression
```

Displaying Changed Lines with sed

With sed, one can use the same functions (searching strings in text) as in grep. The following sed command:

```
$ sed -n '/memory/p' /var/log/messages
...
```

has the same result as

```
$ grep memory /var/log/messages
...
```

Adding p (print) at the end of the sed command causes all lines to print. The -n (no-print) option deactivates this way and the functionality is the same as grep.
When doing replacements in text files, print changed lines, use p again at the end of the expression.

```
$ sed -n 's/2012/2013/gp' Thesis.txt
Due date: 13.02.2013
Measurements made on January 03, 2013.
```

Reusing the Searchstring in the Replacement String

Reusing the search string is sometimes required in the replacement string. This way, prefixing or postfixing to an existing string is possible. The character to use for this is &.

Example:

```
$ echo "/usr/local" | sed 's:/usr/local:&/bin:'
/usr/local/bin
```

As before, use an alternative separator (: instead of /) to make the sed expression more readable.

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Deleting Matching Lines

To delete matching lines use the following syntax:

```
sed /searchpattern/d filename
```

By using `d` **d**elete matching lines in `filename`.
Example: Remove empty lines:

```
$ cat numbers.txt

131 437

4383 8821

2368 2232
$ sed "/^$/d" numbers.txt > numbers2.txt
$ cat numbers2.txt
131 437
4383 8821
2368 2232
```

^$ represents lines that do not have any content (beginning of the line = end of the line).

Deleting Lines Based on Addresses

To target specific lines in a file use the following syntax:

```
sed 'ADDRESS'd filename
```

Use line numbers for `ADDRESS`.
Example: Delete invalid line:

```
$ cat lines.txt
1. line
2. line
0. line
3. line
$ sed 3d lines.txt > lines2.txt
$ cat lines2.txt
1. line
2. line
3. line
```

Specify areas with `,` (comma). This way,

```
sed 1,3d file
```

removes everything starting from the 1st to the 3rd line in `file`.

Overview

1. grep
2. awk
   - Program Structure and Call Syntax
   - Simple and Formatted Output
   - Comparisons
   - Begin and End Actions
   - Variables, Types, and Expressions
     - Predefined Variables
     - User Defined Variables
     - Passing Variables to awk from the Shell
     - String Concatenation
     - Types and Expressions
   - Control Structures
   - Functions
   - Regular Expressions
3. sed
   - Deleting Matching Lines
   - Adding Lines

Adding Lines

The sed syntax for adding lines is:

```
sed 'ADDRESS a\Attachment' File
sed '/PATTERN/ a\Attachment' File
```

Example: Adding enclosing ?> to a PHP file:

```
$ cat beginner.php
<?php
    echo "Hello World!";
$ sed '/;/a\
?>' beginner.php > expert.php
$ cat expert.php
<?php
    echo "Hello World!";
?>
$
```

Instead of a pattern use a line number for addressing as well.
Achieve the same result with the following expression:

```
sed '$ a\
?>'
```

More Information



[IBM] IBM Developer: Awk by example
https://developer.ibm.com/tutorials/l-awk1/



[BADOP] Awk in 20 minutes
https://ferd.ca/awk-in-20-minutes.html